GECG10069 (561084) F26: Introduction to Programming (C++)

Lab 3 : IOStream

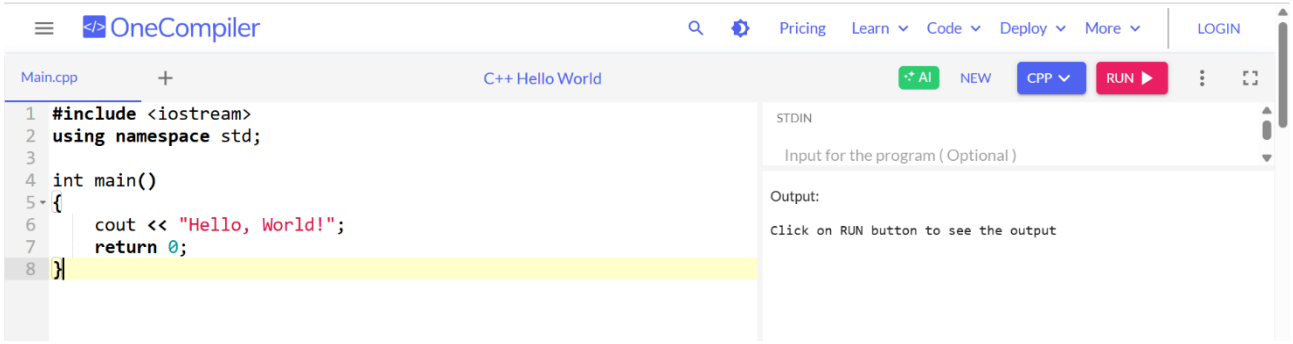


What you will learn from Lab 3

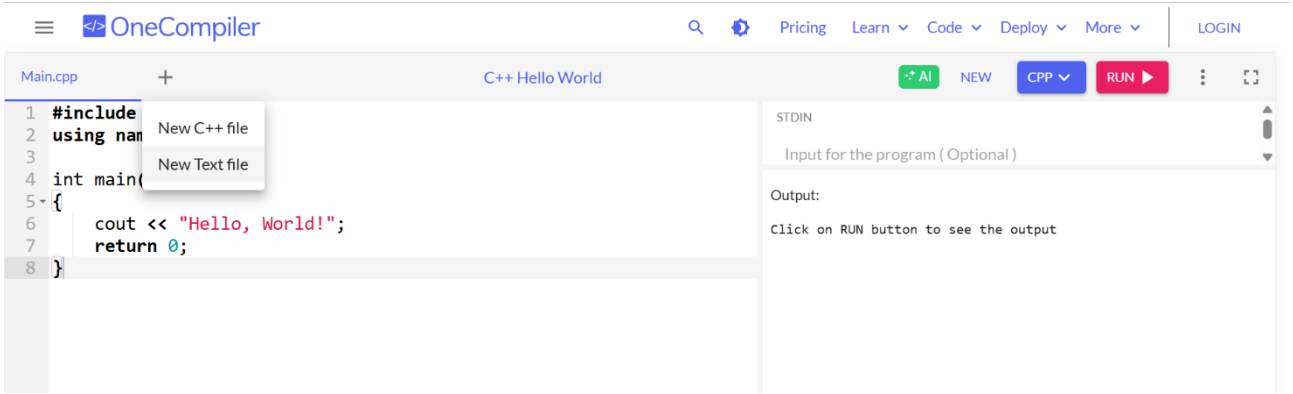
In this laboratory, you will further explore special output symbols and practice using variables in your programs.

TASK 3-1 : HOW TO USE INPUT FILE STREAM

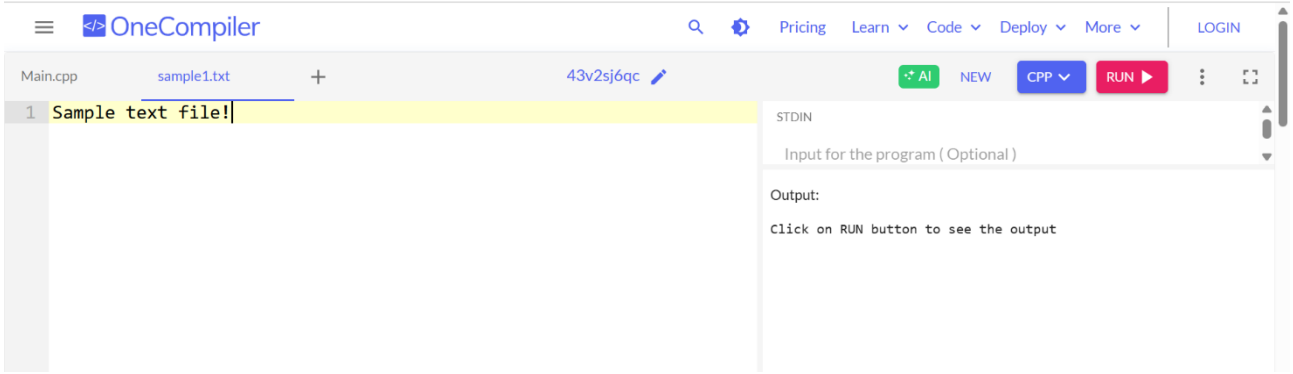
✓ Add an input text file



✓ Select “New Text file”



✓ Filename and content can be modified



✓ Try to read the content in “sample1.txt”

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
    ifstream iff;
    string words;
    iff.open("sample1.txt");
    getline(iff, words);
    cout << words << endl;
    return 0;
}
```

TASK 3-2 : EXAMPLE

✓ Try to mix geteline() and cin

✓ Align output through setw() and setprecision()

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    string subject;
    int score;

    cout << "Enter subject name: ";
    getline(cin, subject);   // 讀科目名稱

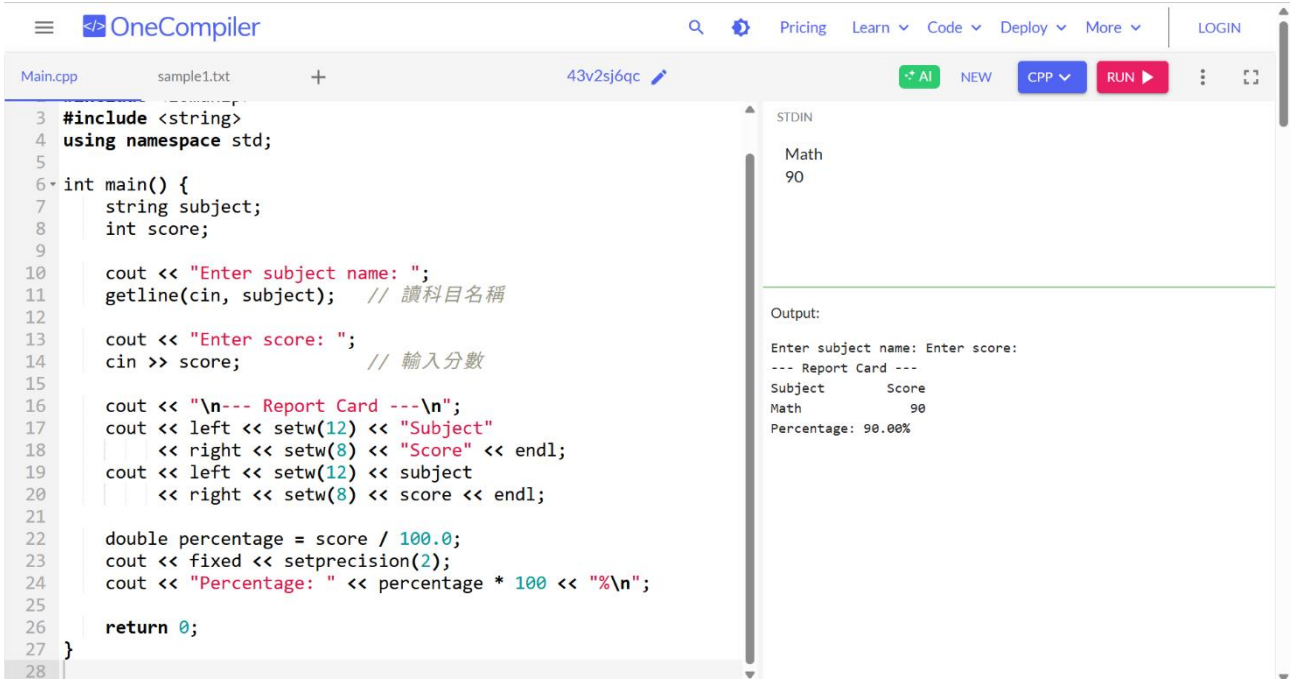
    cout << "Enter score: ";
    cin >> score;            // 輸入分數

    cout << "\n--- Report Card ---\n";
    cout << left << setw(12) << "Subject"
         << right << setw(8) << "Score" << endl;
    cout << left << setw(12) << subject
         << right << setw(8) << score << endl;

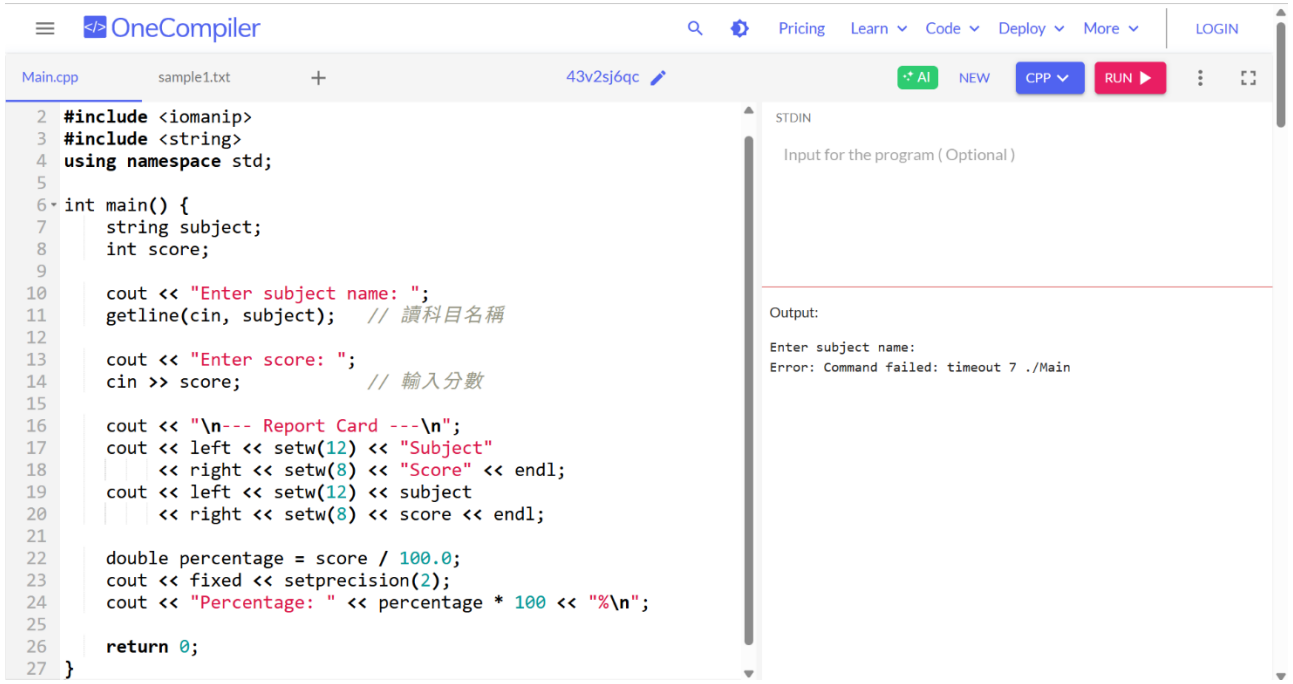
    double percentage = score / 100.0;
    cout << fixed << setprecision(2);
    cout << "Percentage: " << percentage * 100 << "%\n";

    return 0;
}
```

✓ Enter input then press “RUN” button.



✓ If you forget to enter input first, it will take a while and show the following error.



EXERCISE 3-1 : RECTANGEL

Description -

You are given a file named "*data.txt*" with **exactly three lines**.
Each line in the file contains **two integers** (**`int`**) stands for two sides of a rectangle.

For each line,
calculate its **area** and **perimeter** of the rectangle.

Print the results line by line as the sample output provided below.

Formatting Rules -

- This problem **doesn't set a strict output format requirement**. As long as the students generate their output similar to the sample output with correct computation results, the TA can accept the demo as correct.

Constraints -

- $all_the_input_numbers \in$ ***integer*** (**`int`**)
- $calculation_result \in$ ***integer*** (**`int`**)
- $all\ number\ in\ each\ row \neq 0$

Only One Testcase -

data.txt (Please use the function in **<`fstream`>** to read it.)

```
7 3
12 12
25 4
```

Sample Output (You only need to print it (use **`cout`**). Do not export any files.)

```
Area: 21, Perimeter: 20
Area: 144, Perimeter: 48
Area: 100, Perimeter: 58
```

EXERCISE 3-2 : GRADE SHEET

Description -

Read the given **student name** (`string`), **class no.**(`char`), and **year**(`int`), **GPA**(`double`).
Print a small grade sheet with aligned columns and **two-decimal** numbers.

Input (four lines, in this order) -

- **Student name**
- Class no.
- year
- **GPA**

Formatting Rules -

- Header row with columns: Name, Class, Year, GPA
- Use `setw` to align:
  - Name: left, width 20
  - Class: right, width 6
  - Year: right, width 10
  - GPA: right, width 12
- Use `fixed` and `setprecision(2)` for GPA.
- Print exactly the four columns.

Reminder -

- Please notice that the **names** might contain more than one words
- We use `std::cin` and its relevant functions (`ignore()`, `getline()`…) to get all the inputs in this problem

Constraints -

- *0 < year < 7*
- *length(student name) < 18*
- *0 < GPA < 4.3*

Sample Testcases -

Attention:
If the sample input lines contain trailing whitespace (行尾多餘空格) on your end,
please remove it manually to avoid errors between **`cin`** and **`getline()`**

Sample Input - 1

```
Sukuna
D
4
4.25
```

Sample Output - 1

```
Name                 Class       Year         GPA
Sukuna                   D          4        4.25
```

Sample Input - 2

```
Gojo Satoru
A
1
4.26789123
```

Sample Output - 2

```
Name                 Class       Year         GPA
Gojo Satoru              A          1        4.27
```